

Where to Spend Test-Time Compute: Allocation by Irreversibility for a Frozen 120B Agent

Abhishek Shetty

Independent Researcher - Team Rosebud

ashetty21@berkeley.edu

Abstract

In this CAR-bench Track 2 submission, with the model fixed at gpt-oss-120b, all of our engineering goes into the harness that wraps it. Our main finding is negative but useful: nearly every sophisticated test-time-compute mechanism we built for this model was net-neutral or net-negative: scatter-then-sharpen, semantic-agreement mode routing, weak-verifier self-critique, retrieval-based policy injection, value-of-information clarification and deeper consensus all failed to beat a simpler harness. The design that survived allocates compute by irreversibility: reads are free and prefetched, mutations pay for a narrow 2-of-3 consensus, and clarification is rationed to one question. Against the stock baseline under identical roles (Gemini 3.5 Flash as both user simulator and policy judge) it raises Pass³ from 50/101 to 68/101 (+17.8 points) at 1.43 LLM calls per action decision.

1 Overview

CAR-bench Challenge Track 2 asks for a car-assistant agent built on a fixed, competition-supplied model and graded against exact ground-truth action sequences under a simulated user [Kirmayr *et al.*, 2026]. With the model frozen, every lever we have sits in the harness that wraps it. The field’s default answer is to fill that layer with machinery: the model proposes, a deterministic runtime validates and executes, and layers of sampling, refinement, and self-critique are stacked on top to raise reliability [Yao *et al.*, 2023; Shinn *et al.*, 2023; Madaan *et al.*, 2023]. On this frozen, weak model that approach mostly failed. In measurement nearly every added mechanism came out net-neutral or net-negative (Section 5). The principle behind what remained is that compute should be allocated by *irreversibility*, not spread uniformly.

CAR-bench’s reward construction makes actions asymmetric. Extra `get` calls carry no penalty, so information is free. A wrong mutating call is scored against an exact ground-truth action sequence and is not repaired by a later correction, so mutations are irreversible. Pass³ [Yao *et al.*, 2024] multiplies per-trial reliability, so variance at a mutation point is cubed. Finally, a question does not resolve into the environ-

ment but into a simulated user, whose answer is outside the agent’s control.

A harness that spends compute uniformly ignores all three facts. Ours follows them:

- **Reads: spend freely.** Schema-validated prefetch issues the read-only `get` calls *implied* by the request (those whose required arguments are already bound by the user’s words or the prior transcript, resolved against the tool schemas), so the model reasons over observed state instead of guessing it.
- **Mutations: pay for certainty.** At any drafted decision containing a state mutation, the harness samples two additional completions and executes a 2-of-3 (`tool`, `arguments`) signature majority when one exists, otherwise falling back to the original draft.
- **Asks: ration.** At most one harness-injected clarifying question per episode, and every numeric mutation value must trace to the user’s own words, a stored preference actually read or a literal clarification answer.

2 Architecture and compute budget

Figure 1 shows the structure of one agent step. Guards consume no model calls; a guard rejection triggers a re-decision, bounded by an trial-level escalation budget of three. The consensus fan-out issues two additional independent samples only when the draft mutates state and resolves them by a deterministic majority vote. Consequently the chain of *dependent* model calls in a decision is short: a draft, plus one or more guard-triggered re-decisions when guards reject.

3 Harness Mechanisms

Right-sized reasoning budget. At the stock 1,024-token completion cap, gpt-oss-120b [OpenAI, 2025] exhausts its budget on reasoning and returns empty content on hard turns. The stock pipeline reports that empty output to the simulator as an error and then loops on the error. Raising the initial completion cap to 4,096 tokens clears most of these errors.

Truncation rescue. A residual tail of hard turns still stops on the length limit *before* the model emits a usable action. Rather than pass that truncation to the simulator as an error, the harness detects a length-terminated completion and re-issues the *same* decision at most twice, raising the completion cap toward 8,192 tokens, recovering would-be-lost turns.

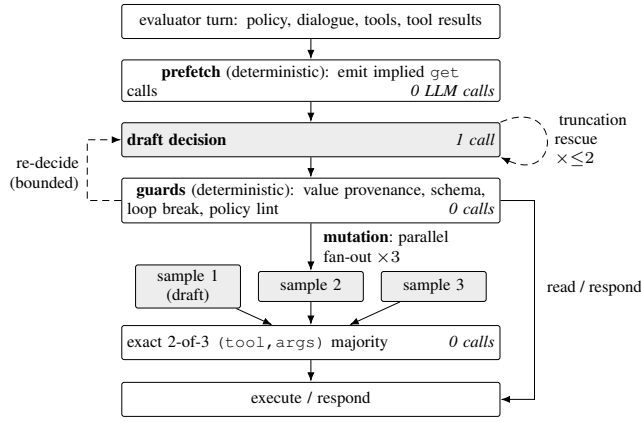


Figure 1: One next-action decision (one baseline agent step). Shaded stages consume model calls. **The dependent LLM-call chain is short:** a draft, plus one or more guard-triggered re-decisions when a guard rejects (retries re-issue a call at the same position; counting each as its own call, the observed maximum over the public tasks is 4). At a mutation the harness draws two additional *independent* samples: a *parallel fan-out* of width 3 whose majority vote is deterministic, so it adds no sequential depth. *Branches:* read/respond decisions bypass consensus; a guard rejection re-routes. *Retry loops* (dashed): a bounded guard-triggered re-decision, and a truncation-rescue (≤ 2 re-issues of a truncated call at the same position). Measured mean: **1.43 calls per action decision**.

Deterministic guard stack. Value provenance is the central guard: a mutation may not invent a numeric value. Four supporting guards run alongside it. Schema and reference preflight checks a drafted call’s arguments against the tool schemas. A loop breaker blocks repeated identical reads. Policy-lint checks the drafted action against the hard constraints in the policy text, such as required-read provenance and disclosure obligations, and rejects or reroutes any violation. A final guard discloses actions the harness declined to execute. All are task-free: they read only the transcript, the supplied tool schemas and the policy text.

Mutation-point consensus. The mechanism is a narrowed self-consistency [Wang *et al.*, 2023]: sampling is restricted to mutation-bearing decisions and agreement is required on the exact canonical (`tool`, `arguments`) signature. The two extra samples run on a copy of the episode state. Across the 303-trial validation run it was invoked 339 times, reached a majority 276 times, and executed an override of the model’s first draft 40 times; the remaining 63 invocations found no majority and fell back to the original draft. Requiring exact-signature agreement rather than semantic clustering matters here: disagreement at a mutation point is usually a genuine parameter dispute, and treating near-matches as agreement would execute one of two materially different actions.

Fixed configuration. The submitted arm runs a single fixed setting: executor reasoning effort *medium*; a 380-token task-independent system prompt; an initial completion cap of 4,096 tokens with a truncation-rescue ladder of at most two re-issues (caps 4,096 and 8,192); an episode-level escalation budget of three guard re-decisions; mutation-point consensus of width 3 (the draft plus two independent samples); an ask

Arm (under Gemini)	Pass ³	Pass ¹	p50/trial
Stock baseline	50/101 (49.5%)	63.0%	2.86s
This work	68/101 (67.3%)	75.9%	3.63s

Table 1: 101-task validation split, $n=3$. Harness effect +17.8 points Pass³ (+18 tasks). The gain is significant by an exact paired McNemar test ($p < 0.001$; 22 tasks won, 4 lost), with a paired-bootstrap [Efron, 1979] 95% CI of [+8.9, +27.7] points.

Family	Baseline		Our harness
	GPT-5-mini	Gemini	Gemini
Base	19/42	24/42	31/42
Hallucination	18/42	24/42	31/42
Disambiguation	6/17	2/17	6/17

Table 2: Pass³ by family. Changing the simulator helps the baseline on base and hallucination but *costs* it two-thirds of its disambiguation score.

budget of one clarifying question per episode; and sampling temperature left at the provider default (nonzero).

4 Results

On a 101-task validation split, under the competition’s Gemini 3.5 Flash [Google DeepMind, 2026] roles, our submitted harness reaches **68/101 Pass³** (67.3%) and 75.9% Pass¹ at a median 3.63 s per trial, against the stock baseline’s 50/101 (49.5%), 63.0%, and 2.86 s (Table 1): a +17.8-point Pass³ gain, significant by an exact paired McNemar test [McNemar, 1947] ($p < 0.001$).

Two models can play the user-simulator and policy-judge roles. During development we used GPT-5-mini [Singh and others, 2026] for cost-efficiency. Before submitting we re-measured our arm and the stock baseline with Gemini 3.5 Flash (Table 1). Throughout this paper, “under GPT-5-mini” and “under Gemini” name which model played those two roles. Comparing harnesses while using the same simulator matters; we see that the stock baseline scores 43/101 under GPT-5-mini but 50/101 under Gemini. 6.9 points of gain are attributable just to the simulator model.

Policy compliance, with a caveat. Among trials that reached policy scoring, our arm recorded **0/164** policy failures against the stock baseline’s **37/177** (20.9%). The denominators are not directly comparable: our arm also produced 16 trials that never reached scoring (non-terminating episodes that exhausted the step budget). The 0/164 is produced mechanically by the guard stack rejecting policy-violating mutations before execution (value provenance, policy lint, disclosure). The non-termination tail is a distinct, fixable failure: a read-spiral the loop-breaker misses because the argument-varying reads are not byte-identical. A per-tool read cap would stop it and the 16 trials are real headroom. Because the submission is frozen we have deliberately not applied the fix.

4.1 The simulator determines the disambiguation problem

The role shift is strongly non-uniform (Table 2). Under Gemini the stock baseline collapses on disambiguation from 6/17 to 2/17 Pass³ while gaining +11 tasks on base and hallucination. Trace inspection explains the inversion: GPT-5-mini frequently leaks the hidden criterion in its opening turn (“turn on the *driver* reading light”; “pick the *shortest* one”), leaving no ambiguity, whereas Gemini withholds it (“turn on the reading lights”) and the task becomes what it was designed to be. This affects the comparability of any CAR-bench result read with a substitute simulator.

4.2 Ablations of the shipped design

Consensus contributes, modestly. We disabled the mutation-point ensemble and held everything else fixed. Under Gemini on a 101-task split, Pass³ falls from 68 to 63 and Pass¹ from 75.9% to 71.9%, the loss concentrated on base (31 → 28) and hallucination (31 → 28) with disambiguation mostly flat (6 → 7). This single $n=3$ read is not individually significant (exact paired McNemar $p = 0.23$; 8 tasks lost, 3 gained). Consensus is a real but narrow contributor.

Spending consensus off the mutation point does not pay. Consensus-on-vs-off shows self-consistency helps somewhere; it does not show that *restricting* it to mutations beats spreading it. We re-ran the shipped arm on the full 101-task validation split under Gemini, changing one flag: consensus applied to respond decisions as well, which is strictly more compute. Terminal consensus lands at 67/101 Pass³ against the shipped 68/101, a wash by an exact paired McNemar test ($p = 1.0$; 7 tasks won, 8 lost), and slightly *lower* Pass¹ (74.6% vs 75.9%), despite spending roughly two extra model calls on every respond turn. Deepening the ensemble *at* the mutation point rather than widening it was likewise flat (Table 3).

Why asks are rationed. The shipped harness caps clarifying questions at one per episode. To test whether that cap is too strict, we ran a counterfactual arm that asks far more freely and compared it to the shipped arm on all 101 tasks. The extra asking helps only when the simulator actually holds the withheld answer: asked to remove a waypoint with no stated route preference, the agent listed concrete options, the simulator revealed “the shortest route”, and a failing task recovered (0/3 → 2/3). When the simulator holds no answer it improvises: on a task whose user never mentioned fan speed, the agent asked, the simulator invented “3”, the ground truth was 1, and a reliably passing task collapsed (3/3 → 0/3). Aggregated, the two sides cancel: the arm scores 67/101 against the shipped 68/101 (5 tasks lost, 4 gained, an exact paired McNemar tie). This is not evidence that asking *hurts*; the finding is the asymmetry in when it can help.

5 Negative results: what the weak model could not use

gpt-oss-120b is weak on this benchmark: in the CAR-bench study it scores a published SCORE of 0.28 against the best system’s 0.54 [Kirmayr *et al.*, 2026]. That capability ceiling

Mechanism	Measured on	Decisive read
<i>Measured ablations (n=3)</i>		
Scatter-then-sharpen	18-task	P ¹ 0.35 (stock 0.56)
Semantic mode-routing	18-task	P ¹ −0.11; P ³ flat
Six-repair failure bundle	101-task	P ³ 60 (ours 68)
Clarification gate	101-task	P ³ 67 (ours 68)
Terminal consensus (respond)	101-task	P ³ 67 (ours 68)
<i>Dev probes (dropped, no measured gain)</i>		
Self-critique gate	n/a	weak self-critic
Retrieval policy injection	n/a	no gain
Native harmony transport	n/a	no gain
Temperature 0	n/a	no gain
Deepen consensus to 5	n/a	no gain (forks persist)

Table 3: Everything we built and dropped, in two tiers. *Top*: five controlled ablations, each with its decisive Pass metric and comparator named in the cell (*stock* = stock baseline; *ours* = the shipped harness); the split in use differs (early 18-task proxy vs. later 101-task). *Bottom*: five mechanisms killed as quick dev probes without a rigorous measured comparison.

explains our results: on this model, added test-time-compute machinery mostly failed. The harness we submit is what remained after removing components ablations did not justify.

The architecture we abandoned. Our first system, scatter-then-sharpen, was the maximal one: scatter many candidate continuations, sharpen them through layered refinement, and adjudicate with a ~4,900-line task-tuned guard layer. On a frozen 18-task proxy at $n=3$ it reached Pass¹ 0.35 against a stock baseline of 0.56 on the same tasks (below doing nothing). More machinery made a weak model *less* reliable: each extra generated candidate is another chance to execute a subtly wrong irreversible mutation, and each refinement pass can rewrite an already-correct draft.

The sophisticated redesign. We replaced scatter-then-sharpen with a literature-grounded design: cluster fast parallel drafts by semantic agreement and *route* on that signal into distinct behavioral modes (commit, deliberate, clarify, verify), backed by a weak-verifier ensemble in place of the guard layer. That arm did win overall (Pass¹ 0.72 on the 18-task proxy; +0.16 over baseline and +0.33 on disambiguation). But a four-arm ablation localized the win to one component. Adding the *ensemble* lifted base Pass³ from 0.50 to 0.67; adding the *routing branches* on top of it was net-neutral on Pass³ and *cost* 0.11 Pass¹. In fact, the *verify* branch fired on 1% of decisions and *clarify* on 22% with no Pass³ lift. The novel part, letting the model decide *which mode to be in*, did nothing, because this model cannot reliably make that judgment. The plain part, sampling a few drafts and acting when they agree, did all the work and is the part we kept.

Other mechanisms that failed. The same pattern held across every remaining mechanism (Table 3). A six-repair bundle hand-targeting known failure families scored 60/101, below the minimal harness’s 68. A weak-verifier self-critique gate failed because this model is not a reliable judge of its own output, consistent with refinement that works when the feedback signal originates *outside* the model [Shinn *et al.*,

2023], and against the intrinsic self-refinement reported on stronger models [Madaan *et al.*, 2023]. Value-of-information-gated clarification, retrieval-based policy injection, a native harmony transport [Mavrin, 2026], temperature 0 and deepening consensus to five samples were each implemented and measured to be net-neutral or net-negative. Contested three-way forks stay contested at five; we find that disagreement marks genuine model uncertainty rather than sampling noise [Li *et al.*, 2026].

Two caveats bound these negatives. Most were measured under GPT-5-mini, and several rest on $n=3$ measurements of a small task set, so we trust them only as within-proxy A/B contrasts (e.g. ensemble vs. ensemble-plus-routing). However, GPT-5-mini leaks the disambiguation criterion (Section 4.1), so the proxy holds little genuine ambiguity, and a mechanism that pays only on ambiguous turns (clarification gating, VoI clarification, the router’s clarify branch) can read null simply because of GPT-5-mini. The two conclusions that most needed confirming off the proxy, that consensus contributes and that spending it off the mutation point does not pay, we re-measured under Gemini (Section 4.2).

The lesson. On a model strong enough to use them, width, self-critique and mode-routing may pay off; ours was not and on it the return was in *narrowing and subtraction*, not addition. These conclusions are scoped to one frozen model (gpt-oss-120b).

6 Compute and rate-limit assumptions

These assumptions are needed to interpret the submission.

Call budget. The rule allows up to five *sequential* LLM calls per baseline step. Over 1,836 baseline steps in a 303-trial run, the agent averaged 1.43 total calls per step; the two consensus samples at a mutation point are parallel calls, so the mean *sequential* depth is 1.06 and 94.6% of steps use a single call. Counting only sequential calls, the deepest chain over the 1,836 steps was 4 (once) and no step reached 5. The trial-level budget of three guard re-decisions could in principle all land on one step, and each draft or re-decision can itself trigger up to two truncation re-issues, a configured worst case of $(1+3) \times 3 = 12$ sequential calls. That case is unreachable in practice, because truncation rescue fires only on a rare length-terminated completion and the re-decision budget is shared across the whole episode rather than allotted per step; hence the observed maximum of 4. Because the submission is frozen we did not add a hard per-step counter; in an unfrozen harness we would cap sequential calls at five directly rather than rely on this empirical headroom.

Token budget. Against the 500k-token average allowance per task, our submission averages $\sim 123k$, about a quarter, split $\sim 116k$ input, $3.3k$ reasoning, $7.0k$ output. A right tail (p95 $\sim 295k$, single-trial max $\sim 491k$) leaves the average comfortable headroom.

Latency and rate limits. The agent induces a median 3.63 s of LLM latency per trial (p90 9.4 s, worst 20.2 s), a 26.8% p50 (34.8% p90) premium over the stock baseline from the prefetch reads and the two consensus samples at

mutation points. Pass³ is the primary metric, so we accept the 0.77 s of added median latency for the +18 tasks it buys.

Measurement discipline. The headline numbers come from a single fixed run of three trials per task, reported as measured rather than the best of several. Two arms we rejected (terminal consensus and the clarification gate) missed pass/fail thresholds we had set before running them, not after seeing the results.

7 Benchmark observations

Inspecting traces surfaced several systematic weaknesses in the benchmark’s scoring and task data. **We document these; we did not exploit them.**

The hallucination family largely grades intervention detection, not capability reasoning. A hallucination task removes a tool or result field from the catalog and its scorer skips vehicle state, tool subset and policy, so the outcome reduces to two conditions: no tool raised an execution error and the user emitted its end token after the agent acknowledged the missing capability. But the intervention is exposed by the altered catalog or by a getter returning the evaluator’s “unknown” sentinel, so a mechanical detector identifies **93/98** rows (94.9%) offline with no capability reasoning. Our harness does not run this detector and does not obtain its advantage incidentally.

Vehicle state is sampled only at user-interaction boundaries. The environment records a state hash only when an assistant turn *begins* with a user-facing action, not after a batch of tool calls, so a mutation that is wrong and corrected before the agent next addresses the user is never scored. Scanning across 963 trials we ran, **75 non-ground-truth intermediate states** survive across **44 passing records**. These are structural counts (what the scorer cannot observe, not proof each state was wrong), but they show intermediate-state grading here measures only endpoints.

Row-level data defects. Fourteen of the 254 tasks (5.5%) have buggy data or labels. The clearest: four navigation answer keys write a route ID straight into the action without the tool reads the policy requires first, so the ground truth breaks the rule the task is testing. These stay passable only because extra reads are free and the scorer checks tool *names*, not arguments. The other ten are minor: two mislabeled disambiguation tasks, one malformed result-removal path, and duplicate action indices.

Single-outcome grading. Exact-match grading is unforgiving in three ways: one failed probe call sinks a task even when the real goal was met and honestly explained; extra adjustments the user explicitly authorized are still marked wrong; and a mistake corrected right away still counts as uncorrected. The balance favors process over outcome.

References

[Efron, 1979] B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

- [Google DeepMind, 2026] Google DeepMind. Gemini 3.5 flash model card. <https://deepmind.google/models/model-cards/gemini-3-5-flash/>, 2026. Model card.
- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, United States, July 2026. Association for Computational Linguistics.
- [Li *et al.*, 2026] Xiaochuan Li, Ryan Ming, Pranav Setlur, Abhijay Paladugu, Andy Tang, Hao Kang, Shuai Shao, Rong Jin, and Chenyan Xiong. Benchmark test-time scaling of general LLM agents. *arXiv preprint arXiv:2602.18998*, 2026.
- [Madaan *et al.*, 2023] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17651.
- [Mavrin, 2026] Borislav Mavrin. In harmony with gpt-oss. *arXiv preprint arXiv:2604.00362*, 2026.
- [McNemar, 1947] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- [OpenAI, 2025] OpenAI. gpt-oss model card. *arXiv preprint arXiv:2508.10925*, 2025.
- [Shinn *et al.*, 2023] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.
- [Singh and others, 2026] Aaditya Singh et al. OpenAI GPT-5 System Card, 2026.
- [Wang *et al.*, 2023] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171.
- [Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- [Yao *et al.*, 2024] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.